

Reibungsfreier Mortar-Kontakt in EdelweissFE

Theorie, Herleitung und Implementierung des Normalkontakts
(Branch `0_mortarcontact`)

Manuel Hradsky

Arbeitsbereich für Festigkeitslehre und Baustatik
Universität Innsbruck

7. August 2026

Zusammenfassung

Dieses Dokument beschreibt Theorie, Herleitung und Implementierung des *reibungsfreien* Mortar-Normalkontakts, wie er auf dem Branch `0_mortarcontact` des Finite-Elemente-Frameworks `EdelweissFE` rechnet. Die Darstellung ist bewusst als in sich geschlossene, paper-artige Referenz angelegt: jede Formel, jeder Algorithmus und jede Konstante wird aus den Primärquellen der Literatur hergeleitet und mit exakter Fundstelle (Autor, Jahr, Gleichung bzw. Abschnitt) belegt, und jeder Baustein wird auf die konkrete Codestelle (`Datei:Zeile`) abgebildet. Die Formulierung beruht auf *dualen* Lagrange-Multiplikatoren zur knotenweisen Entkopplung der Kontaktbedingungen ([Wohlmuth, 2000](#)), einer *segmentbasierten* Integration mit linearer Sub-Zellen-Zerlegung für lineare und quadratische Facetten- und Linienelemente (`CONLINE2/3`, `CONQUAD4/8/9`, `CONTRI3/6`), einer konsistenten Randbehandlung partiell überdeckter Elemente ([Cichosz & Bischoff, 2011](#)) und einer semiglatten Primal-Dual Active-Set-Strategie (PDASS) zur Lösung der diskreten Signorini-Bedingungen ([Hüeber & Wohlmuth, 2005](#); [Gitterle et al., 2010](#)). Besonderes Gewicht liegt auf den vier technischen Kernpunkten der zugehörigen Präsentation: (i) warum die Segmentquadratur den Genauigkeitsgrad 5 verwendet (7-Punkt-Dreiecksregel nach [Dunavant 1985](#)), (ii) Herleitung der Koppelmatrizen $\mathbf{D}$, $\mathbf{M}$ und des gewichteten Spalts, (iii) warum die Basistransformation quadratischer Elemente den Faktor $\frac{1}{3}$ trägt ([Popp et al., 2012](#)), und (iv) die Herleitung der Normal-NCP-Funktion ([Gitterle et al., 2010](#), Gl. (55)). Reibung und statische Kondensation sind auf diesem Branch *nicht* enthalten und werden hier bewusst nicht behandelt.

Inhaltsverzeichnis

Geltungsbereich und Implementierungsstand	4
1 Einleitung und Problemstellung	5
2 Kontinuierliche Formulierung	6
2.1 Kinematik und Spalt	6
2.2 Hertz–Signorini–Moreau / KKT-Bedingungen	6
2.3 Schwache Form und Multiplikatorraum	6
3 Diskretisierung und Programmarchitektur	7
3.1 Kontaktelemente als geometrische Rand-Overlays	7
3.2 Verarbeitungs-Pipeline	7
3.3 Modulzuordnung	7
4 Geometrische Verarbeitung: Nachbarsuche, Projektion, Clipping	8
4.1 Nachbarsuche mit Bounding-Volume-Hierarchie	8
4.2 Hilfsebene und Projektion	8
4.3 Sutherland–Hodgman-Clipping und Triangulierung	8
5 Segmentbasierte Quadratur: warum Genauigkeitsgrad 5	9
5.1 Notwendigkeit der segmentbasierten Integration	9
5.2 Anhebung des Integrationsgrades	9
5.3 Die 7-Punkt-Regel vom Grad 5 (Dunavant, 1985)	10
5.4 Der zweidimensionale Fall	10
5.5 Auswertung an den Gauß-Punkten	10
6 Koppelmatrizen D, M und gewichteter Spalt	11
6.1 Entstehung von D und M	11
6.2 Gewichteter (schwacher) Normalspalt	11
6.3 Zeilensummen-Identität und Impulserhaltung	11
7 Duale Lagrange-Multiplikatoren und Biorthogonalität	13
7.1 Definition und Motivation	13
7.2 Diagonalisierung von D	13
7.3 Elementlokale Konstruktion	13
8 Duale Basis für quadratische Elemente: die Basistransformation T_e	14
8.1 Verlust der Integral-Positivität bei quadratischen Formfunktionen	14
8.2 Basistransformation der Formfunktionen	14
8.3 Wahl des Transformationsparameters α	15
8.4 Auswirkung auf die Struktur von D	16
9 Quadratische Facetten: lineare Sub-Cells	17
9.1 Notwendigkeit der linearen Zerlegung	17
9.2 Die konkrete Zerlegung	17
9.3 Wie die Krümmung dennoch eingeht	17
10 Knotennormalen und eingefrorene Geometrie	18
10.1 Flächengewichtete Knotennormale	18
10.2 Einfrorene Geometrie (staggered update)	18

11 Signorini als semismooth NCP	19
11.1 Von den KKT-Bedingungen zur NCP-Funktion	19
11.2 Der Aktivierungsindikator und die Codeform	19
11.3 Der Parameter c_n	20
11.4 PDASS als semismooth Newton	20
12 Lösungsstrategie: Newton-Ablauf und Active-Set	21
12.1 Sattelpunkt-Struktur	21
12.2 Ein Newton pro Increment mit eingefrorener Geometrie	21
12.3 Active-Set-Iteration und Anti-Cycling	21
13 Verifikation	22
14 Forschung: Warum ist hex20 im Ausziehversuch steifer als hex8?	23
14.1 Beobachtung	23
14.2 Der entscheidende Hinweis: die Steifigkeit bestimmt der Kontakt, nicht das Volumenelement	23
14.3 Der physikalische Mechanismus: Lasteinleitung am Bolzenkopf	23
14.4 Abgleich mit der Verifikation	24
14.5 Schlussfolgerung	24
15 Zusammenfassung	25
16 Anwendungsanleitung: Kontaktspezifikation in der Input-Datei	26
16.1 Syntax-Struktur des Mortar-Constraint-Blocks	26
16.2 Schlüsselwörter	26
16.3 Vorbereitung der Oberflächen und Kontaktelemente	26

Geltungsbereich und Implementierungsstand

Beschrieben wird ausschließlich der Zustand auf Branch `0_mortarcontact`: der reibungsfreie Mortar-Normalkontakt in **Sattelpunkt-Formulierung** (die Lagrange-Multiplikatoren sind explizite zusätzliche Unbekannte). Es gibt auf diesem Branch:

- **keine** Coulomb-Reibung (kein `friction_coefficient`-Schlüsselwort),
- **keine** statische/duale Kondensation (kein `condensation`-Schlüsselwort). Die Constraint-Argumente sind `nonMortarSurface`, `mortarSurface`, `field` sowie der NCP-Parameter `cn` (`mortarcontact.py:89-108`); `cn` ist optional (Standard 10^6), sollte aber gesetzt werden (Abschnitt 11).

Auf diesem Branch wird pro Slave-Knoten *genau ein skalarer* Lagrange-Multiplikator instanziiert – die Normalkomponente λ_I der Kontakttraktion (der Kontaktdruck): es gibt einen Multiplikator-Freiheitsgrad je Slave-Knoten, *nicht* `dim` (`mortarcontact.py:348-351`). Die Kontaktkraft entsteht daraus als $\lambda_I \mathbf{n}_I$, also allein in Richtung der Knotennormale (`mortarcontact.py:825, 833`). In der allgemeinen (kontinuierlichen) Mortar-Formulierung ist der Multiplikator ein vollständiger Traktionsvektor $\mathbf{z}_I \in \mathbb{R}^{\dim}$, dessen Tangentialkomponenten erst mit Coulomb-Reibung physikalisch aktiv würden (dann `dim` Multiplikatoren je Knoten). Diese Vektor-Erweiterung ist der natürliche Reibungspfad, auf Branch `0_mortarcontact` aber *nicht* instanziiert. Der Code ist reines Python im Framework `EdelweissFE`; die drei zentralen Quelldateien – `mortarcontact.py` (867 Z.), `mortar_geom_utils.py` (119 Z.) und `contactelement/element.py` (511 Z.) – werden mit ihren vollständigen Pfaden in Abschnitt 3 (Tab. Modulzuordnung) aufgeführt.

Notationskonvention. In der Literatur und in der Präsentation heißt die Slave–Master-Koppelmatrix $\mathbf{M}$. Im Quelltext heißt dieselbe Matrix `C`, während der Bezeichner `M` dort für die lokale Biorthogonalitäts-Massenmatrix reserviert ist. Dieses Dokument folgt der Literatur und schreibt durchgehend $\mathbf{D}$ (Slave–Slave) und $\mathbf{M}$ (Slave–Master); an den Codestellen wird jeweils auf den Bezeichner `C` hingewiesen.

1 Einleitung und Problemstellung

Zwei deformierbare Körper – im Zielanwendungsfall ein Stahllanker (Master) in einem Betonblock (Slave) – sollen an ihrer gemeinsamen Kontaktfläche γ_c Kräfte übertragen, ohne sich zu durchdringen. Die Finite-Elemente-Netze der beiden Körper sind *nichtkonform*: ihre Knoten treffen an der Kontaktfläche nicht aufeinander. Ein klassischer Knoten-auf-Knoten-Kontakt (*node-to-node*) setzt die Nichtdurchdringung punktweise an paarenden Knoten durch und ist damit auf nichtkonforme Netze nicht anwendbar: es gibt keine natürliche Knotenpaarung, und ein naiver Knoten-auf-Segment-Kontakt (*node-to-segment*) verletzt den Kontakt-Patch-Test, weil er einen konstanten Druck nicht exakt überträgt (Wriggers, 2006).

Die **Mortar-Methode** setzt die Kontaktbedingung stattdessen *schwach* durch: nicht punktweise, sondern im integralen Mittel über die Kontaktfläche, gewichtet mit den Formfunktionen der Slave-Seite. Der zugehörige Lagrange-Multiplikator λ ist physikalisch die Kontakttraktion (im reibungsfreien Fall der Kontaktdruck) und lebt auf der Slave-Fläche. Werden für λ *duale* Formfunktionen im Sinne von Wohlmuth (2000) gewählt, so entkoppeln die Kontaktbedingungen knotenweise (Abschnitte 6 und 7), und die Methode besteht den Kontakt-Patch-Test bei optimaler Konvergenzordnung. Die Mortar-Methode ist heute der Standardzugang für Kontakt zwischen nichtkonformen Netzen bei großen Deformationen (Puso & Laursen, 2004; Popp et al., 2010; Farah, 2018).

Dieses Dokument beschreibt die konkrete reibungsfreie Umsetzung in EdelweissFE. Die Darstellung folgt der Verarbeitungs-Pipeline: von der Geometrie (Nachbarsuche, Projektion, Clipping, Quadratur) über die diskreten Koppelmatrizen und den gewichteten Spalt bis zur Lösung der Kontaktbedingung als semiglattes Komplementaritätsproblem.

2 Kontinuierliche Formulierung

2.1 Kinematik und Spalt

Der Slave-Körper trägt den Index (1), der Master-Körper den Index (2). Zu einem Slave-Punkt $\mathbf{x}^{(1)} \in \gamma_c$ wird über die Kontaktnormale $\mathbf{n}$ der nächste Master-Punkt $\hat{\mathbf{x}}^{(2)}$ (die *Projektion*) bestimmt. Der Normalspalt ist

$$g_n = -\mathbf{n} \cdot (\mathbf{x}^{(1)} - \hat{\mathbf{x}}^{(2)}), \quad (1)$$

mit der Vorzeichenkonvention $g_n > 0$ bei geöffnetem Kontakt (Trennung) und $g_n < 0$ bei Durchdringung (Wriggers, 2006; Farah, 2018).

2.2 Hertz–Signorini–Moreau / KKT-Bedingungen

Der reibungsfreie Normalkontakt fordert Nichtdurchdringung, nichtklebende (nur drückende) Kontakttraktion und Komplementarität. Mit dem Kontaktdruck $p = \lambda \geq 0$ (Normalkomponente der Traktion) lauten die Karush–Kuhn–Tucker-Bedingungen (Hertz–Signorini–Moreau)

$$g_n \geq 0, \quad p \geq 0, \quad p g_n = 0 \quad \text{auf } \gamma_c. \quad (2)$$

Entweder ist der Spalt offen ($g_n > 0$) und der Druck null, oder es besteht Kontakt ($g_n = 0$) mit $p \geq 0$ (Wriggers, 2006, Kap. 4).

2.3 Schwache Form und Multiplikatorraum

Die Kontaktbedingung wird über einen Lagrange-Multiplikator $\boldsymbol{\lambda}$ (die Kontakttraktion) in das Prinzip der virtuellen Arbeit eingebracht. Der zusätzliche Kontaktbeitrag zur virtuellen Arbeit ist

$$\delta W_c = \int_{\gamma_c} \boldsymbol{\lambda} \cdot (\delta \mathbf{u}^{(1)} - \delta \mathbf{u}^{(2)}) \, d\gamma, \quad (3)$$

und die schwache Nichtdurchdringung wird mit Testfunktionen $\delta \boldsymbol{\lambda}$ aus dem Multiplikatorraum M als Variationsungleichung angesetzt. Die Wahl des diskreten Raums für $\boldsymbol{\lambda}$ ist entscheidend: Wohlmuth (2000) zeigt, dass ein *dualer* Raum (biorthogonal zu den Slave-Formfunktionen) die inf-sup/LBB-Stabilität sichert und eine optimale a-priori-Fehlerabschätzung liefert, während er zugleich die spätere knotenweise Elimination erlaubt. Genau dieser duale Raum wird hier verwendet (Abschnitt 7). Die Diskretisierung von (1)–(2) führt auf die gewichteten Größen der folgenden Abschnitte: die Koppelmatrizen $\mathbf{D}, \mathbf{M}$, den gewichteten Spalt $\tilde{g}_I$ und die knotenweise NCP-Bedingung.

3 Diskretisierung und Programmarchitektur

3.1 Kontaktelemente als geometrische Rand-Overlays

EdelweissFE besitzt keine eingebauten Kontaktelemente. Der Mortar-Kontakt definiert sie daher selbst als rein *geometrische* Rand-Overlay-Elemente (kein Materialkern): sie tragen die Verschiebungs-Freiheitsgrade der zugehörigen Volumenoberfläche und liefern Formfunktionen N_I , duale Funktionen Φ_I und Quadraturpunkte. Erzeugt werden sie aus den Seitenflächen (Sidesets) der Volumenelemente (z. B. CONQUAD8 auf der Oberfläche eines hex20).

Typ	Knoten	Geometrie	Ordnung
CONLINE2 / CONLINE3	2 / 3	2D-Kante	linear / quadratisch
CONQUAD4 / CONQUAD8 / CONQUAD9	4 / 8 / 9	3D-Fläche	linear / quadratisch
CONTRI3 / CONTRI6	3 / 6	3D-Dreieck	linear / quadratisch

3.2 Verarbeitungs-Pipeline

Die Berechnung folgt einer festen Pipeline, die zugleich die Gliederung dieses Dokuments ist:

1. **Nachbarsuche** (BVH) → Kandidatenpaare (Abschnitt 4).
2. **Projektion + Clipping + Triangulierung** → Integrationszellen (Abschnitt 4), für quadratische Facetten nach linearer Sub-Cell-Zerlegung (Abschnitt 9).
3. **Segmentquadratur** (Grad 5) → Koppelmatrizen $\mathbf{D}, \mathbf{M}$ (Abschnitte 5, 6).
4. **Gewichteter Spalt** $\tilde{g}_j$ und **NCP** → Active Set (Abschnitte 6, 11).
5. **Assemblierung + Newton** → Lösung (Abschnitt 12).

3.3 Modulzuordnung

Datei	Rolle
constraints/mortarcontact.py	Segmentierung, $\mathbf{D}, \mathbf{M}$ -Assemblierung, gewichteter Spalt, NCP/Active-Set, K/PExt-Assemblierung
constraints/mortar_geom_utils.py	Ebenenprojektion, Sutherland–Hodgman-Clipping, Triangulierung, 1D-Clip
elements/contactelement/element.py	Formfunktionen, Quadraturpunkte, Basistransformation $\mathbf{T}_e$, lokale $\mathbf{M}_e/\mathbf{D}_e/\mathbf{A}_e$

4 Geometrische Verarbeitung: Nachbarsuche, Projektion, Clipping

Bei nichtkonformen Netzen ist a priori unbekannt, welche Master-Facetten welche Slave-Facetten überlappt. Die geometrische Verarbeitung bestimmt diese Überlappungen und zerlegt sie in Integrationszellen (Folien 5–6).

4.1 Nachbarsuche mit Bounding-Volume-Hierarchie

Eine naive Prüfung aller Slave–Master-Paare kostet $\mathcal{O}(N^2)$. Stattdessen werden die achsparallelen Hüllboxen (AABB) der Master-Facetten in einer *Bounding-Volume-Hierarchie* organisiert – einem binären Baum mit Median-Split entlang der längsten Achse und Blattgröße ≤ 2 (Ericson, 2004). Pro Slave-Facetten liefert eine Baumabfrage nur wenige *Kandidaten*; der Aufwand sinkt auf $\sim \mathcal{O}(N \log N)$. Die AABB erhalten einen Sicherheitsrand $\max(0.15 \cdot \text{Ausdehnung}, 0.1)$; nur für die Kandidaten wird anschließend die exakte geometrische Verschneidung durchgeführt.

4.2 Hilfsebene und Projektion

Für ein Kandidatenpaar wird an der Slave-(Sub-)Facette eine *Hilfsebene* definiert (Stützpunkt $\mathbf{p}_0$, Normale $\mathbf{n}$). Beide Polygone – Slave und Master – werden in diese Ebene projiziert und dort in ebenen Koordinaten dargestellt (`mortar_geom_utils.py`, `get_tangent_basis`, `to_plane_coords`). Diese Ebene dient nur der Schnittbildung; die tatsächliche Geometrie geht später über die Newton-Rückabbildung an den Gauß-Punkten ein (Abschnitt 5).

4.3 Sutherland–Hodgman-Clipping und Triangulierung

In der Ebene wird das Master-Polygon gegen das (konvexe) Slave-Polygon geschnitten. Dafür dient der *Sutherland–Hodgman-Algorithmus* (Sutherland & Hodgman, 1974): das zu schneidende Polygon wird sukzessive an jeder Kante des Clip-Polygons gekappt (*reentrant polygon clipping*). Der Algorithmus setzt ein *konvexes* Clip-Polygon voraus – deshalb wird als Clip-Fenster die (konvexe) Slave-Facetten bzw. Sub-Cell verwendet, nicht umgekehrt. Das Ergebnis ist das konvexe Überlappungspolygon $\text{Slave} \cap \text{Master}$ (`sutherland_hodgman_clip`).

Da die Segmentquadratur (Abschnitt 5) auf Dreiecken definiert ist, wird das Überlappungspolygon abschließend *fächerartig* von einer Ecke aus trianguliert (`triangulate_polygon`): ein n -Eck ergibt $n - 2$ Integrationsdreiecke. Weil das Polygon konvex ist, ist diese Triangle-Fan-Zerlegung immer gültig. Im 2D-Fall reduziert sich das Ganze auf einen 1D-Intervallschnitt (`clip_1d_segments`). Das Vorgehen entspricht MOOSEs `AutomaticMortarGeneration`.

5 Segmentbasierte Quadratur: warum Genauigkeitsgrad 5

Nach dem Clipping (Abschnitt 4) liegt die Überlappung eines Slave–Master-Paars als Menge ebener Integrationsdreiecke (3D) bzw. Teilstrecken (2D) vor. Auf diesen wird integriert. Dieser Abschnitt beantwortet die zentrale Frage der Präsentation (Folie 7): *Warum verwendet der Code Genauigkeitsgrad 5 – konkret die 7-Punkt-Dreiecksregel im 3D-Fall?* Die Antwort hat drei Schichten: (i) warum überhaupt *segment*basiert integriert wird, (ii) warum der effektive Integrationsgrad über das nominelle Formfunktionsprodukt angehoben werden muss, und (iii) woher die konkrete 7-Punkt-Regel stammt.

5.1 Notwendigkeit der segmentbasierten Integration

Die Mortar-Matrizen $\mathbf{D}, \mathbf{M}$ (Abschnitt 6) enthalten unter dem Integral ein Produkt aus Größen *beider* Seiten – der dualen bzw. Standard-Formfunktionen der Slave-Seite und den auf die Slave-Fläche zurückprojizierten Formfunktionen der Master-Seite. Bei nichtkonformen Netzen besitzt dieser Integrand *Knicke* (schwache Diskontinuitäten) im Inneren einer Slave-Facette: nämlich überall dort, wo eine Master-Elementkante die Slave-Facette kreuzt. Farah et al. (2015) formulieren das explizit:

“the integrand represents a non-smooth function which cannot be evaluated exactly by using standard Gauss rules. This non-smoothness stems from the locally supported Lagrange polynomials on master and slave side having kinks at the respective element nodes and edges.” (Farah et al., 2015, S. 214)

Eine Standard-Gauß-Regel, die über die ganze Facette gelegt wird (*element*based), “sieht” diese Knicke nicht und ist deshalb hochgradig empfindlich gegenüber Gauß-Punkt-Zahl und Netz-Verhältnis (Farah et al., 2015, S. 217). Der Ausweg ist die *segment*basierte Integration: die Überlappung wird in glatte, knickfreie Zellen mit konstantem Jacobian zerlegt (hier: Dreiecke), auf denen der Integrand wieder polynomial-glatt ist und Gauß-Quadratur ihre volle Ordnung entfaltet (Farah et al., 2015, Abschn. 3.1). Genau das leistet die Clipping–Triangulierungs-Pipeline aus Abschnitt 4.

5.2 Anhebung des Integrationsgrades

Innerhalb einer knickfreien Integrationszelle bleibt ein Resteffekt: die Rückabbildung eines Gauß-Punkts von der Hilfsebene in die *Natur*koordinaten von Slave und Master ist *nichtlinear* (bei gekrümmten/verzerrten Facetten und generell durch die Projektion). Dadurch ist der Integrand – als Funktion der Zell-Koordinaten – kein Polynom vom Grad des reinen Formfunktionsprodukts, sondern effektiv höhergradig. Farah et al. (2015) geben deshalb die Faustregel und die konkrete Empfehlung an (der Satz findet sich wortgleich in Farah 2018, Anh. A.1.1):

“the highest polynomial degree that can be computed exactly by the integration rule should be chosen somewhat higher than the product of shape functions on slave and master side would indicate. This is due to the nonlinearity of the projections between auxiliary plane and actual contact surfaces. Based on our experiences, we suggest to use seven Gauss points per triangular integration cell, which are able to exactly calculate a polynomial degree of 5.” (Farah et al., 2015, S. 217; identisch Farah, 2018, Anh. A.1.1, S. 209–210)

Die Wahl “7 Punkte / Grad 5” ist also eine *empirisch abgesicherte* Empfehlung: sie liegt im gesättigten Genauigkeitsbereich, höhere Punktzahlen ändern die Lösung praktisch nicht mehr (Farah, 2018, Kap. 6). Im Code ist genau das umgesetzt (`mortarcontact.py:201–224`), mit demselben Begründungskommentar und Verweis auf Farah (2018, Anh. A.1.1).

5.3 Die 7-Punkt-Regel vom Grad 5 (Dunavant, 1985)

Die symmetrische 7-Punkt-Regel mit Genauigkeitsgrad 5 auf dem Referenzdreieck stammt aus [Dunavant \(1985\)](#). Er approximiert (Gl. (4), S. 1129)

$$\int_A f(\alpha, \beta, \gamma) dA = A \sum_{i=1}^{n_g} w_i f(\alpha_i, \beta_i, \gamma_i), \quad \alpha + \beta + \gamma = 1, \quad (4)$$

in baryzentrischen Koordinaten und bestimmt die $(w_i, \alpha_i, \beta_i, \gamma_i)$ aus den Momentenbedingungen $\int_A \alpha^i \beta^j dA = 2A i! j! / (i+j+2)!$ (Gl. (13), S. 1132). Die Punkte werden in *Symmetrieorbits* gruppiert: ein Schwerpunkt (n₀), Dreiergruppen auf den Seitenhalbierenden (n₁, je 3 Punkte) und allgemeine Sechsergruppen (n₂), mit Gesamtzahl $n_g = n_0 + 3n_1 + 6n_2$ (Gl. (34), S. 1135). Für Grad $p = 5$ liefert Dunavant $n_0 = 1, n_1 = 2, n_2 = 0$, also

$$n_g = 1 + 3 \cdot 2 = 7 \quad (\text{Dunavant, 1985, Tab. III/IV, S. 1135–1137}). \quad (5)$$

Das ist effizienter als die Gauß-Produktregel, die für Grad 5 neun Punkte bräuchte ($7 < 9$). Die konkreten Werte ([Dunavant, 1985, Anh. II, S. 1140](#)) sind:

Orbit	Gewicht w_i (pro Punkt)	α	β, γ	#Punkte
Schwerpunkt	0.225000000000	0.333333333333	0.333333333333	1
n_1 -Orbit 1	0.132394152789	0.059715871790	0.470142064105	3
n_1 -Orbit 2	0.125939180545	0.797426985353	0.101286507323	3

Die Dreiergruppen entstehen durch zyklische Permutation von (α, β, γ) ; die Gewichtssumme ist $0.225 + 3 \cdot 0.132394 \dots + 3 \cdot 0.125939 \dots = 1.000$. Zwei Eigenschaften machen gerade diese Regel für den Kontakt attraktiv: **alle sieben Punkte liegen im Inneren** des Dreiecks ($\alpha, \beta, \gamma \in (0, 1)$) und **alle Gewichte sind positiv**. Grad 5 ist oberhalb von Grad 2 die niedrigste Dunavant-Regel mit diesen beiden Eigenschaften – die Grad-3-Regel etwa hat ein negatives Gewicht (-0.5625 ; [Dunavant, 1985, S. 1140](#)) und ist damit numerisch ungünstig. Dieselbe Regel steht im Code in `mortarcontact.py:201–224`; die Gauß-Punkte sind identisch, die Gewichte dort $(9/80, (155 \mp \sqrt{15})/2400, \text{Summe } \frac{1}{2} \text{ statt } 1)$ sind bereits mit der Referenzdreiecksfläche $\frac{1}{2}$ verrechnet, sodass das Produkt $w \cdot J_{\text{cell}}$ und damit die Regel unverändert bleibt.

5.4 Der zweidimensionale Fall

Im 2D-Fall wird pro geclipptem Liniensegment eine 3-Punkt-Gauß-Legendre-Regel auf $[-1, 1]$ mit Punkten $\xi \in \{0, \pm\sqrt{0.6}\}$ und Gewichten $\{5/9, 8/9, 5/9\}$ eingesetzt (`mortarcontact.py:192–198`). Sie ist exakt bis Polynomgrad $2n - 1 = 5$ (Nullstellen des Legendre-Polynoms P_3) – daher der Kommentar “degree 5” im Code, konsistent mit der Grad-5-Wahl im 3D-Fall. *Bemerkung:* [Farah \(2018\)](#) verwendet im 2D-Kontext empirisch *fünf* Gauß-Punkte pro Segment, nicht drei. Beide Wahlen folgen derselben Logik (Grad über das Formfunktionsprodukt anheben wegen nichtlinearer Projektion); der Code priorisiert hier die einheitliche *Grad-5*-Aussage über 2D und 3D. Für die im 2D auftretenden Integranden ist Grad 5 in der Praxis ausreichend (bestätigt durch die 2D-Patch-Tests, Abschnitt 13).

5.5 Auswertung an den Gauß-Punkten

An jedem Gauß-Punkt wird der Ebenenpunkt per *Newton-Iteration* in die Naturkoordinaten von Slave und Master zurückabgebildet, sodass dort N_s und N_m (und die dualen Φ) ausgewertet werden können; das gewichtete Produkt geht mit $w \cdot J_{\text{cell}}$ in die Summation über alle Zellen ein und liefert D und M (`mortarcontact.py:600–648`; Summenform nach [Farah et al. 2015, Alg. 1, S. 215](#)):

$$D_{IK} = \sum_{c=1}^{n_{\text{cell}}} \sum_{g=1}^{n_g} w_g \Phi_I(\xi_g^{(1)}) N_K^{(1)}(\xi_g^{(1)}) J_c, \quad M_{IJ} = \sum_c \sum_g w_g \Phi_I(\xi_g^{(1)}) N_J^{(2)}(\xi_g^{(2)}) J_c. \quad (6)$$

6 Koppelmatrizen $\mathbf{D}$, $\mathbf{M}$ und gewichteter Spalt

Dieser Abschnitt leitet die diskreten Mortar-Koppelmatrizen und den gewichteten Spalt her – die Größen der Präsentation (Folie 8), die am Ende der Segmentquadratur stehen.

6.1 Entstehung von $\mathbf{D}$ und $\mathbf{M}$

Man diskretisiert die Kontakttraktion mit dem dualen Multiplikatorfeld $\boldsymbol{\lambda}_h = \sum_j \Phi_j \mathbf{z}_j$ (allgemeine Vektorform; auf Branch `0_mortarcontact` wird davon nur die Normalkomponente $\mathbf{z}_j \cdot \mathbf{n}_j = \lambda_j$ instanziiert, s. Geltungsbereich) und die Geometrie beider Seiten mit den jeweiligen Standard-Formfunktionen $N^{(1)}, N^{(2)}$ und setzt beides in die Kontakt-Virtuelle-Arbeit ein. Sortiert man nach den Knotenbeiträgen um, treten genau zwei Matrizen auf – die *Mortar-Matrizen* (Farah, 2018, Gl. (3.34)–(3.36); Popp et al., 2012, Gl. (3.5)–(3.6); Popp et al., 2009, Gl. (35)/(36)):

$$\mathbf{D}[j, k] = D_{jk} \mathbf{I} = \int_{\gamma_{c,h}^{(1)}} \Phi_j N_k^{(1)} d\gamma \mathbf{I}, \quad \mathbf{M}[j, l] = M_{jl} \mathbf{I} = \int_{\gamma_{c,h}^{(1)}} \Phi_j (N_l^{(2)} \circ P_h) d\gamma \mathbf{I}. \quad (7)$$

Dabei ist j ein Slave-Multiplikator-knoten, k ein Slave-Verschiebungsknoten (1), l ein Master-Verschiebungsknoten (2), $\mathbf{I} \in \mathbb{R}^{\dim \times \dim}$ die Einheitsmatrix, und $P_h : \gamma_{c,h}^{(1)} \rightarrow \gamma_{c,h}^{(2)}$ die diskrete Kontaktabbildung (die Projektion). Beide Integrale laufen über die *Slave-Fläche*. $\mathbf{D}$ (Slave–Slave) ist quadratisch, $\mathbf{M}$ (Slave–Master) im Allgemeinen rechteckig (Popp et al., 2012, S. B427). Die algebraische Kontaktkraft ist $\mathbf{f}_c = [\mathbf{0} \quad -\mathbf{M} \quad \mathbf{D}]^T \mathbf{z}$ (Farah, 2018, Gl. (3.38)). Im Code werden $\mathbf{D}, \mathbf{M}$ in `mortarcontact.py:487–690` assembliert (die Slave–Master-Matrix heißt dort `c`; vgl. Notationskonvention).

6.2 Gewichteter (schwacher) Normalspalt

Die diskrete schwache Nichtdurchdringung wird pro Slave-Knoten j zu einem *gewichteten Spalt* (Farah, 2018, Gl. (3.40))

$$\tilde{g}_{n,j} = \int_{\gamma_{c,h}^{(1)}} \Phi_j g_{n,h} d\gamma. \quad (8)$$

Setzt man $g_{n,h}$ über (1) ein und sortiert nach Knotenpositionen um, erhält man die Positionsform (Gitterle et al., 2010, Gl. (36) und (50); Popp et al., 2009, Gl. (50)):

$$\tilde{g}_j = \mathbf{n}_j \cdot \left(\sum_k \mathbf{D}[j, k] \mathbf{x}_k^{(1)} - \sum_l \mathbf{M}[j, l] \mathbf{x}_l^{(2)} \right) = \mathbf{n}_j \cdot \left(\mathbf{D}[j, j] \mathbf{x}_j^{(1)} - \sum_l \mathbf{M}[j, l] \mathbf{x}_l^{(2)} \right), \quad (9)$$

wobei die zweite Gleichheit gilt, weil $\mathbf{D}$ bei dualer Basis diagonal wird (Abschnitt 7). Genau diese Form steht im Code (`mortarcontact.py:785–795`, Bezeichner `c` für $\mathbf{M}$).

Warum schwach/nodal statt punktweise? Der gewichtete Spalt ist ein volumetrisches (integrales) Maß, das dennoch als knotenweises Näherungsmaß verwendet wird (Farah, 2018: “*The discrete weighted gap $\tilde{g}_{n,j}$ represents a volumetric measure ... Nevertheless, it is utilized as node-wise measure*”). Der tiefere Grund für die schwache Auswertung liegt in der Ungleichungsnatur der Nichtdurchdringung in Kombination mit Formfunktionen höherer Ordnung: eine punktweise (starke) Durchsetzung ist damit nicht konsistent behandelbar (Popp et al., 2012, S. B429). Die knotenweisen HSM-Bedingungen lauten dann $\tilde{g}_{n,j} \geq 0$, $\lambda_{n,j} \geq 0$, $\lambda_{n,j} \tilde{g}_{n,j} = 0$ (Popp et al., 2012, Gl. (3.7); Farah, 2018, Gl. (3.41)) – die diskrete Fassung von (2).

6.3 Zeilensummen-Identität und Impulserhaltung

Eine zentrale Konsistenzeigenschaft ist die Zeilensummen-Identität

$$\sum_k \mathbf{D}[j, k] = \sum_l \mathbf{M}[j, l] \quad \forall j \quad (10)$$

(Farah, 2018, Gl. (4.99)). Sie folgt aus der Erhaltung des linearen Impulses $\mathbf{f}_\lambda^{(1)} - \mathbf{f}_\lambda^{(2)} = \mathbf{0}$: mit der Partition-of-Unity der Formfunktionen $\sum_k N_k^{(1)} = \sum_l N_l^{(2)} = 1$ (Farah, 2018, Gl. (4.97)) reduziert sich die Differenz der Zeilensummen auf $\int \Phi_j d\gamma - \int \Phi_j d\gamma = 0$, was *immer* erfüllt ist, sofern $\mathbf{D}$ und $\mathbf{M}$ über *dasselbe* diskrete Gebiet integriert werden (Farah, 2018, Gl. (4.100)–(4.101); identisch Popp et al. 2010 und Puso & Laursen 2004). Physikalisch sichert (10) die Translationsinvarianz des Spalts (eine starre Translation beider Körper erzeugt keinen künstlichen Gap) und ist Voraussetzung für das Bestehen des Kontakt-Patch-Tests. Genau deshalb müssen $\mathbf{D}$ und $\mathbf{M}$ im Code über *dieselben* Integrationszellen summiert werden (die konsistente Randbehandlung, Abschnitt 9, stellt das auch für partiell überdeckte Elemente sicher).

7 Duale Lagrange-Multiplikatoren und Biorthogonalität

7.1 Definition und Motivation

Die dualen Formfunktionen Φ_j sind durch die *Biorthogonalität* zu den Slave-Formfunktionen definiert (Wohlmuth, 2000, Gl. (3.5); Popp et al., 2012, Gl. (4.1); Popp et al., 2009, Gl. (30); Farah, 2018, Gl. (3.46)):

$$\int_{\gamma_{c,h}^{(1)}} \Phi_j N_k^{(1)} d\gamma = \delta_{jk} \int_{\gamma_{c,h}^{(1)}} N_k^{(1)} d\gamma. \quad (11)$$

Wohlmuth (2000) führte diese dualen Räume ein, um bei gleicher *Optimalität* (inf-sup/LBB-Stabilität mit h -unabhängiger Konstante, optimale a-priori-Fehlerabschätzung der Ordnung $\mathcal{O}(h)$ in der Energienorm) lokal getragene Multiplikator-Basisfunktionen zu erhalten:

“this Lagrange multiplier space is replaced by a dual space without losing the optimality of the method ... all the basis functions of the new method are supported in a few elements. The mortar map can be represented by a diagonal matrix; in the standard mortar method a linear system of equations must be solved.” (Wohlmuth, 2000, Abstract)

7.2 Diagonalisierung von $\mathbf{D}$

Setzt man (11) in die Definition (7) von $\mathbf{D}$ ein, verschwinden alle Außerdiagonalterme:

$$D_{jk} = \int \Phi_j N_k^{(1)} d\gamma = \delta_{jk} \int N_k^{(1)} d\gamma, \quad \implies \quad \mathbf{D} = \text{diag} \left(\int N_k^{(1)} d\gamma \right). \quad (12)$$

Die gesamte algebraische Struktur nimmt damit die Gestalt eines Knoten-auf-Segment-Kontakts an, und $\mathbf{D}$ ist trivial invertierbar (Farah, 2018, Abschn. 3.4.1.2, S. 33: “the algebraic form of the slave side mortar matrix $\mathbf{D}$... becomes a diagonal matrix”). Auf diese Diagonalität stützt sich der Projektionsoperator $\mathbf{P} = \mathbf{D}^{-1} \mathbf{M}$ (Farah, 2018, Gl. (3.65)) – die algebraische Grundlage jeder späteren Kondensation. *Anmerkung:* Diese statische Kondensation ist auf Branch `0_mortarcontact` bewusst *nicht* aktiviert (Sattelpunkt-Formulierung); die Diagonalität von $\tilde{\mathbf{D}}$ wird hier nur für die Normierung im NCP-Indikator (Abschnitt 11) genutzt.

7.3 Elementlokale Konstruktion

Bei Elementen mit nicht-konstanter Jacobi-Determinante werden die Φ_j als Linearkombination der Standard-Formfunktionen dargestellt, $\Phi_j = c_{jk} N_k$, mit der Koeffizientenmatrix (Farah, 2018, Gl. (3.53)–(3.56))

$$\mathbf{A}_e = \mathbf{D}_e \mathbf{M}_e^{-1}, \quad (\mathbf{D}_e)_{jk} = \delta_{jk} \int_e N_k J de, \quad (\mathbf{M}_e)_{jk} = \int_e N_j N_k J de. \quad (13)$$

Hier sind $\mathbf{D}_e, \mathbf{M}_e$ *lokale Hilfs-Massenmatrizen* zur Konstruktion der dualen Basis – nicht zu verwechseln mit den globalen Koppelmatrizen $\mathbf{D}, \mathbf{M}$ (Farah unterscheidet sie im Symbolverzeichnis). Im Code ist (13) zweifach umgesetzt: als Referenz-Fallback in `element.py:470-508` (`A_e = D_e @ inv(M_e) @ T_e`) und – bevorzugt – zur Laufzeit über die *echte* Segmentquadratur (`mortarcontact.py:650-673`), sodass die Biorthogonalität auf dem tatsächlichen Integrationsgebiet erfüllt ist (konsistente Randbehandlung nach Cichosz & Bischoff (2011), Abschnitt 9). Der Zusatzfaktor $\mathbf{T}_e$ ist die Basistransformation für quadratische Elemente (Abschnitt 8); für lineare Elemente ist $\mathbf{T}_e = \mathbf{I}$.

8 Duale Basis für quadratische Elemente: die Basistransformation T_e

Für quadratische Kontaktelemente ist die direkte duale Basiskonstruktion aus Abschnitt 7 nicht unmittelbar brauchbar. Dieser Abschnitt begründet dies, leitet die Abhilfe – eine Basistransformation – her und diskutiert die Wahl des Transformationsparameters α einschließlich der begründeten Abweichung von der Literaturempfehlung (vgl. Präsentation, Folie 9).

8.1 Verlust der Integral-Positivität bei quadratischen Formfunktionen

An die Multiplikator-Formfunktionen wird die *Integral-Positivität* gestellt (Popp et al., 2012, Gl. (4.2)):

$$\int_{\text{ele}} \Phi_j d\gamma > 0. \quad (14)$$

Der Grund ist direkt algorithmisch: Ist (14) verletzt, kann ein physikalisch offener Spalt einen *negativen* gewichteten Spalt ergeben (und umgekehrt); die Active-Set-Strategie behandelt dann aktive Knoten als inaktiv und umgekehrt und konvergiert nicht (Popp et al., 2012: “the numerical algorithm will generate nonphysical gaps and penetrations . . . a nonconverging active set strategy”).

Genau das tritt bei quadratischen Formfunktionen auf. Das Eckknoten-Integral ist nicht positiv (Popp et al., 2012, Gl. (4.3)/(4.4)):

$$\int_{\text{ele}} N_1^{\text{quad8}} d\gamma < 0, \quad \int_{\text{ele}} N_1^{\text{tri6}} d\gamma = 0. \quad (15)$$

Ein anschauliches, numerisch greifbares Beispiel liefert Puso et al. (2008, Gl. (29)): zwei ebene, *nicht* durchdrungene Serendipity-Flächen im Abstand l erzeugen am Eckknoten den gewichteten Spalt $g_A = Al/3 > 0$ – also eine *vorgetäuschte* Durchdringung, allein wegen der Nichtpositivität der quadratischen Funktion. (Ein wörtlicher Wert “ $-1/12$ ” als negatives *Gewicht* kommt in der Literatur nicht vor; der im Code-Docstring genannte Zahlenwert bezieht sich auf das negative Eckknoten-Integral im Sinne von Popp et al. (2012, Gl. (4.3)).)

8.2 Basistransformation der Formfunktionen

Statt der originalen Formfunktionen wird eine transformierte Basis verwendet (Popp et al., 2012, Gl. (4.5)):

$$\tilde{N} = T_e N, \quad (16)$$

deren T_e jede *Kantenmittelnknoten*-Zeile mit dem Anteil α auf die beiden benachbarten Eckknoten umverteilt und $1 - 2\alpha$ auf sich behält; die Eckknotenzeilen bleiben die Identität. Für CONQUAD8 lautet die Matrix (mit $\alpha = \frac{1}{3}$) explizit

$$T_e = \begin{bmatrix} 1 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & \frac{1}{3} \\ 0 & 1 & 0 & 0 & \frac{1}{3} & \frac{1}{3} & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & \frac{1}{3} & \frac{1}{3} & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{3} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{1}{3} \end{bmatrix} \quad (\text{element.py:424-468}). \quad (17)$$

Die Transformation hat drei nachweisbare Eigenschaften (Popp et al., 2012, S. B431): sie ist *symmetrisch* (jeder Kantenknoten trägt gleich viel zu seinen beiden Eck-Nachbarn bei), sie erhält die *Partition of Unity* ($\sum_j \tilde{N}_j = 1$), und sie ist durch $\alpha < \frac{1}{2}$ nach oben beschränkt, damit auch

die Kantenknoten-Integrale positiv bleiben. Aus der transformierten Basis werden dann – wie bei linearen Elementen über (13) – die dualen Funktionen gebildet.

Die Transformation wird nur auf die *serendipity*-Typen mit nichtpositivem Eckintegral angewandt (CONQUAD8, CONTRI6; CONLINE3). Für das voll-Lagrange'sche CONQUAD9 (aus hex27) entfällt sie: dessen Eckknoten-Integrale sind bereits strikt positiv ($\frac{1}{9}$ auf dem Referenzquadrat), und das Kanten-zu-Ecken-Schema erfasst den zusätzlichen Zentrums-knoten ohnehin nicht – für CONQUAD9 wird daher wie für lineare Typen die Identität verwendet (`element.py:454-462`).

8.3 Wahl des Transformationsparameters α

Die *Struktur* der Transformation (symmetrische Kanten-zu-Ecken-Umverteilung, Erhaltung der Partition of Unity, obere Schranke $\alpha < \frac{1}{2}$) geht auf Lamichhane et al. (2005) zurück, die die duale Basis für Elemente höherer Ordnung durch eine *elementweise Umverteilung* von Kanten-Mittelknoten auf die Eckknoten konstruieren (Lamichhane et al., 2005, Gl. (3.10)/(3.12)); Popp et al. (2012) verallgemeinern dies auf die Finite-Deformation. Der *Zahlenwert* von α ist damit noch nicht festgelegt; die Literatur bietet mehrere begründete Kandidaten:

- *Integral-Positivität* (14) verlangt im unverzerrten Fall $\alpha > 0$ (tri6) bzw. $\alpha > \frac{1}{8}$ (quad8).
- Die zusätzliche exakte Reproduktion linearer Polynome (P_1) im dualen Raum ergibt (unverzerrt) $\alpha = \frac{1}{12}$ (tri6) bzw. $\alpha = \frac{1}{5}$ (quad8). Genau diese Werte leiten Lamichhane et al. (2005) aus Biorthogonalität und P_1 -Reproduktion her: den Faktor $\frac{1}{12}$ für quadratische Tetraeder (Gl. (3.10)) und $\frac{1}{5}$ für die Serendipity-Hexaeder, deren Facette gerade das quad8-Element ist (Gl. (3.12)).
- Popp et al. (2012) empfehlen für quadratische *Flächenelemente* $\alpha = \frac{1}{5}$, das nach ihrer Analyse die Integral-Positivität für alle praktisch auftretenden Elementverzerrungen sichert (Popp et al., 2012, S. B431).
- Farah (2018) wählt für tet10-*Volumenelemente* $\alpha_q = \frac{1}{3}$ (Farah, 2018, Gl. (6.20), S. 164), begründet durch die Schranke $\alpha_q < \frac{1}{2}$, Integral-Positivität und numerische Erfahrung.

Die vorliegende Implementierung verwendet $\alpha = \frac{1}{3}$ (`element.py:451`) und weicht damit bewusst von Pops flächenspezifischer Empfehlung $\frac{1}{5}$ ab. Diese Wahl ist nicht kosmetisch, sondern durch eine Eigenschaft der *segmentbasierten* Auswertung begründet, die im Folgenden dargelegt wird.

Integral-Positivität genügt bei partieller Überdeckung nicht. Pops Garantie bezieht sich auf die Positivität des *Voll-Element*-Integrals $\int_{\text{ele}} \tilde{N}_j d\gamma$. In der segmentbasierten Formulierung (Abschnitt 9) werden die dualen Koeffizienten und die knotenweisen Gewichte $\tilde{D}_{jj}$ jedoch über die *tatsächliche Überlappungsfläche* $\gamma_c^{\text{ovl}} \subseteq \text{ele}$ integriert (konsistente Randbehandlung nach Cichosz & Bischoff (2011)). Ist ein Slave-Element nur *teilweise* in Kontakt, so ist das relevante Gewicht $\int_{\gamma_c^{\text{ovl}}} \tilde{N}_j d\gamma$; dessen Positivität für *beliebige* Teilgebiete ist genau dann gesichert, wenn die transformierte Formfunktion nicht nur ein positives Integral besitzt, sondern *punktweise nichtnegativ* ist,

$$\tilde{N}_j(\xi) \geq 0 \quad \forall \xi \in \text{ele}, \quad (18)$$

eine strikt stärkere Forderung als (14). Für die serendipity-Eckfunktion existiert eine Schwelle α^* , oberhalb derer (18) gilt; sie liegt zwischen den beiden Literaturwerten $\frac{1}{5}$ und $\frac{1}{3}$.

Numerischer Befund. Dies ist im Verifikationslauf direkt belegt (Abschnitt 13, Tests 05 und 08):

- **Test 05 (partielle Überdeckung, CONQUAD8):** Bei um 0.3 verschobenem Master (70% Überdeckung) sind mit $\alpha = \frac{1}{3}$ alle knotenweisen Gewichte strikt positiv; mit $\alpha = \frac{1}{5}$

wird ein Eckknoten-Gewicht *negativ* (≈ -0.0049) – $\tilde{N}_{\text{Ecke}}$ ist dort also nicht punktweise nichtnegativ.

- **Test 08 (verzerrter hex20-Patch-Test):** Der übertragene Kontaktdruck weicht mit $\alpha = \frac{1}{3}$ punktuell um 14.3% vom Sollwert ab (innerhalb der 20%-Toleranz für verzerrte Netze), mit $\alpha = \frac{1}{5}$ dagegen um 21.8% (außerhalb der Toleranz). Die übertragene *Gesamtkraft* ist in beiden Fällen exakt.

Der Wert $\alpha = \frac{1}{3}$ liegt mit größerem Abstand zur unteren Positivitätsschranke und macht die transformierten Eckfunktionen – im untersuchten Bereich – punktweise nichtnegativ, sodass die Gewichte auch bei Teilkontakt und Netzverzerrung positiv und die diskrete Signorini-Bedingung robust bleiben. Der Preis ist der Verzicht auf die exakte Reproduktion linearer Polynome, die $\alpha = \frac{1}{5}$ auf unverzerrten Elementen böte; für die Konsistenzordnung des Kontakts ist dies unkritisch, und die unverzerrten Patch-Tests werden mit beiden Werten exakt bestanden.

Bemerkung. Da im Zielanwendungsfall (Ausziehversuch eines Ankers im Betonblock) verzerrte Netze und über den Lastverlauf wechselnder Teilkontakt auftreten, ist die durch $\alpha = \frac{1}{3}$ gesicherte punktweise Nichtnegativität hier die ausschlaggebende Eigenschaft – sie hat Vorrang vor der exakten Linearreproduktion von $\alpha = \frac{1}{5}$. Dies deckt sich mit Farahs Begründung “numerische Erfahrung” (Farah, 2018, Gl. (6.20)).

8.4 Auswirkung auf die Struktur von $\mathbf{D}$

Mit der Transformation ist die globale Matrix $\mathbf{D}$ *nicht* mehr diagonal. Popp et al. (2012, Gl. (4.12)) zeigen aber, dass sie sich exakt in zwei trivial invertierbare Faktoren spaltet:

$$\mathbf{D} = \tilde{\mathbf{D}} \mathbf{T}^{-1}, \quad \tilde{D}_{jk} = \int \Phi_j \tilde{N}_k d\gamma \text{ diagonal}, \quad \mathbf{D}^{-1} = \mathbf{T} \tilde{\mathbf{D}}^{-1}. \quad (19)$$

Alle Vorteile der dualen Basis bleiben also erhalten, *ohne* $\mathbf{D}$ zu “lumpen” (diagonal zu approximieren). Ein naives Lumping wäre hier falsch: es verletzte die Zeilensummen-Identität (10) bzw. die Konsistenzbedingung $\tilde{D}_{kk} = \sum_j M_{kj}$ und damit den Patch-Test (dieser Konsistenz-Zusammenhang ist bei Cichosz & Bischoff (2011, Abschn. 4.1) belegt). Deshalb wird $\mathbf{D}$ im Code voll (nicht-gelump) assembliert (mortarcontact.py:740–749).

9 Quadratische Facetten: lineare Sub-Cells

9.1 Notwendigkeit der linearen Zerlegung

Der Clipping-Algorithmus (Abschnitt 4) schneidet Polygone in einer *ebenen* Hilfsfläche und ist nur für *gerade* Kanten korrekt. Eine quadratische Facette (CONQUAD8/9, CONTRI6) ist im Allgemeinen gekrümmt und hat quadratisch interpolierte Kanten. [Puso et al. \(2008\)](#) lösen das, indem die gekrümmte Facette in *linear interpolierte* Sub-Segmente zerlegt wird, auf die der Schnitt-Algorithmus unverändert anwendbar ist:

“our approach to geometrically approximate these quadratic surfaces will be to subdivide them into linearly interpolated contact sub-segments ... The algorithms described ... for intersection of linearly defined element surfaces can then be applied almost without modification. In so doing, we of course sacrifice any type of geometrically exact description of the quadratic contacting surfaces in defining the integration algorithm.” ([Puso et al., 2008](#), S. 560–561)

9.2 Die konkrete Zerlegung

Die Zerlegung entspricht exakt [Puso et al. \(2008, Abb. 3\)](#): das Serendipity-Element (quad8) wird in *vier lineare Eck-Dreiecke und ein zentrales Quadrilateral* zerlegt (Abb. 3(e)/(f)), quad9 in vier Quads, tri6 in vier lineare Dreiecke (Abb. 3(c)/(d)). Im Code steht dies als `SUB_CELL_MAP` (`mortarcontact.py:156–183`):

Elementtyp	Sub-Cells (lokale Knotenindizes)
CONQUAD8	[0, 4, 7], [4, 1, 5], [5, 2, 6], [7, 6, 3] und Mittel-Quad [4, 5, 6, 7]
CONQUAD9	[0, 4, 8, 7], [4, 1, 5, 8], [8, 5, 2, 6], [7, 8, 6, 3]
CONTRI6	[0, 3, 5], [3, 4, 5], [3, 1, 4], [5, 4, 2]
CONLINE3	[0, 2], [2, 1] (2D-Analogon)

Die Knoten 4–7 (bzw. 3–5) sind die Kantenmittelnknoten; die Sub-Cells werden über sie aufgespannt.

9.3 Wie die Krümmung dennoch eingeht

Wesentlich: die Zerlegung betrifft *nur* das Integrationsgebiet (die Clip-Geometrie), *nicht* die Feldinterpolation. Über affine Abbildungen zwischen Sub-Segment- und Eltern-Element-Parameterraum ([Puso et al., 2008](#), Gl. (31)/(32)) werden an den Gauß-Punkten weiterhin die *vollen* quadratischen Formfunktionen ausgewertet:

“it is possible to evaluate higher-order shape function products in the integrand of the mortar matrices and the weighted gap without any algorithmic changes. This approach only affects the integration domain itself, which is less accurate in terms of geometry.” ([Farah, 2018](#), Anh. A.1.4, S. 215; Verweis auf [Puso et al. 2008](#))

Die Krümmung geht also ausschließlich über die Formfunktions-Auswertung an den (per Newton rückabbildeten) Gauß-Punkten ein, nie über die geraden Clip-Kanten. Dieses Vorgehen ist inhaltlich identisch zu MOOSEs `AutomaticMortarGeneration` (“Linearize secondary face elements”). Damit wird der hex20-Patch-Test exakt bestanden (Abschnitt 13).

10 Knotennormalen und eingefrorene Geometrie

10.1 Flächengewichtete Knotennormale

Der gewichtete Spalt (9) und die Projektion benötigen eine eindeutige Normale pro Slave-Knoten. Sie wird als *flächengewichtetes Mittel* der angrenzenden Facettennormalen gebildet und normiert (mortarcontact.py:402-458):

$$\mathbf{n}_I = \frac{\sum_{f \ni I} \mathbf{n}_f}{\left\| \sum_{f \ni I} \mathbf{n}_f \right\|}, \quad \mathbf{n}_f = \frac{1}{2}(\mathbf{v}_1 - \mathbf{v}_0) \times (\mathbf{v}_2 - \mathbf{v}_0) \quad (20)$$

(im 2D: $\mathbf{n} = [t_y, -t_x]$ aus der Kantentangente). Diese gemittelte Normale garantiert ein stetiges Normalenfeld über Facettengrenzen hinweg, was für die Konsistenz des Kontakts an Elementkanten nötig ist.

10.2 Eingefrorene Geometrie (staggered update)

Normalen $\mathbf{n}_I$ und Koppelmatrizen $\mathbf{D}, \mathbf{M}$ werden *einmal* zu Increment-Beginn in der aktuellen (deformierten) Konfiguration berechnet und dann für alle Newton-Iterationen des Increments festgehalten (mortarcontact.py:738). Dadurch ist die assemblierte Steifigkeit die *exakte* Jacobi-Matrix des Residuums bei fixierter Geometrie – Voraussetzung für die Newton-Konvergenz (Abschnitt 12). Der Preis ist eine gestaffelte (staggered) Behandlung der Geometrienichtlinearität, die durch hinreichend kleine Lastschritte kontrolliert wird.

Bekannte Einschränkung. An *scharfen* Kanten (z. B. Übergang Ankerschaft→Ankerkopf im hex20-Netz) ist die gemittelte Knotennormale mehrdeutig ($\sim 45^\circ$), was die Projektion dort verschlechtern kann. Für die hier betrachteten glatten Kontaktflächen ist das unkritisch.

11 Signorini als semismooth NCP

Die diskreten Kontaktbedingungen (2) sind Ungleichungen mit Fallunterscheidung (Kontakt / offen). Dieser Abschnitt fasst sie in *eine* nichtglatte Gleichung – die NCP-Funktion (Folie 11) – und begründet ihre Lösung per Active-Set/semismooth Newton.

11.1 Von den KKT-Bedingungen zur NCP-Funktion

Die knotenweisen Hertz–Signorini–Moreau-Bedingungen $\tilde{g}_j \geq 0$, $\lambda_{n,j} \geq 0$, $\lambda_{n,j}\tilde{g}_j = 0$ (Farah, 2018, Gl. (3.41); Gitterle et al., 2010, Gl. (38)) lassen sich mit einem beliebigen $c_n > 0$ äquivalent als *eine* nichtglatte Komplementaritätsfunktion (NCP) schreiben. Die Urform stammt aus Hieber & Wohlmuth (2005, Gl. (3.9)); in der Kontakt-Notation lautet sie (Gitterle et al., 2010, Gl. (55); Farah, 2018, Gl. (3.58))

$$C_{n,j}(\lambda_{n,j}, \mathbf{d}) = \lambda_{n,j} - \max(0, \lambda_{n,j} - c_n \tilde{g}_j) = 0, \quad c_n > 0. \quad (21)$$

Äquivalenz zur KKT-Form. Mit der elementaren Identität $a - \max(0, a - b) = \min(a, b)$ ist (21) genau die *min*-NCP-Funktion

$$C_{n,j} = 0 \iff \min(\lambda_{n,j}, c_n \tilde{g}_j) = 0 \iff [\lambda_{n,j} \geq 0, \tilde{g}_j \geq 0, \lambda_{n,j} \tilde{g}_j = 0], \quad (22)$$

also exakt die drei KKT-Bedingungen – und zwar *unabhängig* vom Wert c_n (Gitterle et al., 2010, Gl. (56): erfüllt (38) “*independently of the choice of the parameter c_n* ”). (Die min/max-Identität ist Standard; eine Fischer–Burmeister-Formulierung wird in den Quellen nicht verwendet und hier nicht benötigt.)

Wer und warum. Die Normal-NCP wurde für Kleindeformation von Hieber & Wohlmuth (2005) eingeführt und von Popp et al. (2009) auf finite Deformation übertragen – dort erscheint sie im reibungsfreien Fall in genau dieser Form (Popp et al., 2009, Gl. (56)); Gitterle et al. (2010) fassen dies in Gl. (55) zusammen. Die mathematische Wurzel der später genutzten Äquivalenz “Active-Set = semismooth Newton” ist Hintermüller et al. (2002) (Satz 3.1: lokal superlineare, bei starker Semismoothness sogar q -quadratische Konvergenz).

11.2 Der Aktivierungsindikator und die Codeform

Aktiv/inaktiv wird über das Vorzeichen des Arguments der max-Funktion entschieden (Hieber & Wohlmuth 2005, Gl. (3.13) bzw. Gitterle et al. 2010, Gl. (70) – letztere äquivalent als komplementäre *Inaktiv*-Menge mit ≤ 0 geschrieben):

$$\text{Knoten } j \text{ aktiv} \iff s_{n,j} := \lambda_{n,j} - c_n \tilde{g}_j > 0. \quad (23)$$

Im Code steht diese Bedingung mit einer *zusätzlichen Normierung* mit $\tilde{D}_{jj}^{-1}$ (`mortarcontact.py`: 797-839):

$$s_{n,j} = p_{n,j} - c_n \tilde{D}_{jj}^{-1} \tilde{g}_j, \quad \text{aktiv} \iff s_{n,j} > 0, \quad (24)$$

mit dem physikalischen Druck $p_{n,j} = -\lambda_j \operatorname{sgn}(D)$. *Bemerkung:* der Faktor $\tilde{D}_{jj}^{-1}$ ist *nicht* Teil der Originalgleichung (55)/(3.58) – dort steht $c_n \tilde{g}_j$ mit dem *gewichteten* Spalt direkt. Da $\tilde{g}_j$ ein volumetrisches, über $\tilde{D}_{jj}$ skaliertes Maß ist (Abschnitt 6), macht die Division durch $\tilde{D}_{jj}$ den Term $c_n \tilde{D}_{jj}^{-1} \tilde{g}_j$ *druckartig* und damit dimensionell vergleichbar mit $p_{n,j}$. Es ist eine bewusste, dimensionell konsistente *Normierungsentscheidung* der Implementierung (im Einklang mit der D^{-1} -Skalierung der dualen Mortar-Methode), kein Wortzitat aus Gitterle. Die Struktur der NCP und der Aktivierungsindikator (23) sind dadurch unberührt.

11.3 Der Parameter c_n

$c_n > 0$ ist ein rein *algorithmischer* Parameter: bei Konvergenz ist entweder $\tilde{g}_j = 0$ (aktiv) oder $\lambda_{n,j} = 0$ (inaktiv), sodass c_n aus $\min(\lambda_{n,j}, c_n \tilde{g}_j)$ herausfällt – er beeinflusst *die Lösung nicht*, nur das Konvergenzverhalten (Gitterle et al., 2010, S. 565: “*purely algorithmic parameters with no influence on the accuracy of results*”; Farah, 2018, S. 38). Für die Größenordnung wird

$$c_n \sim \mathcal{O}(E) \quad (25)$$

empfohlen (Popp et al., 2009, S. 1367: “ c_n is suggested to be at the order of Young’s modulus E of the contacting bodies”; Farah, 2018, S. 38; Gitterle et al., 2010, S. 565: Parameter “in the range of material parameters”). Der Grund ist eine Skalenbalance: $\lambda_{n,j}$ ist druckartig ($\sim E$ -Dehnung), $\tilde{g}_j$ längen-/volumenartig; $c_n \sim E$ (bzw. nach der Normierung (24) $c_n/\tilde{D}_{jj} \sim E$) bringt beide auf dieselbe Skala. Zu *großes* c_n destabilisiert dagegen die Active-Set-Iteration: Gitterle et al. (2010, Tab. I, S. 565) zeigt für $c_n \in \{10^5, 10^6\}$ mehrfache Aktiv-Mengen-Wechsel (*Chattering*, dort mit * markiert) bis hin zur Nichtkonvergenz, während ein breites mittleres Band robust in wenigen Schritten konvergiert. Im Code ist c_n mit $\tilde{D}_{jj}$ normiert und in der Größenordnung von E gewählt (mortarcontact.py:92-108), mit demselben Chattering-Argument im Kommentar.

11.4 PDASS als semismooth Newton

Die max-Funktion in (21) ist *semismooth*; ihre verallgemeinerte Ableitung ist $\partial_x \max(a, x) = \{0 \text{ falls } x \leq a, 1 \text{ falls } x > a\}$ (Gitterle et al., 2010, Gl. (64)). Damit ist die Primal-Dual Active Set Strategy (PDASS) ein *semismooth Newton-Verfahren*: das Active Set wird in jeder Iteration aus (23) neu bestimmt, das resultierende lineare System (bei fixiertem Set glatt) gelöst, und iteriert, bis sich das Set nicht mehr ändert (Stopp-Kriterium, Hübner & Wohlmuth, 2005, S. 3153; Farah, 2018, Abschn. 3.5.2, “re-interpreted as a semi-smooth Newton method”). Die abstrakte Grundlage dieser Interpretation und Konvergenz liefert Hintermüller et al. (2002) (Satz 3.1); den ersten *konsistent* linearisierten Newton-Ansatz für den dualen Mortar-Kontakt mit PDASS setzte Popp et al. (2009, Gl. (65)/(67)) um. Die beobachtete Konvergenz ist charakteristisch: solange das Set noch wechselt, sinkt das Residuum nur langsam; *sobald das Set fixiert ist, konvergiert das Verfahren quadratisch* – als Folge der konsistenten Linearisierung (Gitterle et al., 2010, S. 560; Farah, 2018, Tab. 4.1, mit Residuenabfall $\sim 10^{-3} \rightarrow 10^{-8} \rightarrow 10^{-11}$ nach Fixierung). Die Implementierung bestätigt das Set-Stabilitäts-Kriterium und sichert es zusätzlich per Anti-Cycling (Bland-Regel) gegen Zustandswiederholung ab (mortarcontact.py:841-867; siehe Abschnitt 12).

12 Lösungsstrategie: Newton-Ablauf und Active-Set

12.1 Sattelpunkt-Struktur

Auf Branch `0_mortarcontact` sind die Lagrange-Multiplikatoren *explizite* zusätzliche Unbekannte (ein skalarer Normalmultiplikator pro Slave-Knoten, `mortarcontact.py:348-351`). Das globale System ist damit ein *Sattelpunkt*-System

$$\begin{bmatrix} \mathbf{K} & \mathbf{B}^\top \\ \mathbf{B} & \mathbf{0} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{d} \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} \mathbf{r}_d \\ \mathbf{r}_\lambda \end{bmatrix}, \quad \mathbf{B} = [\mathbf{D} \quad -\mathbf{M} \quad \mathbf{0}], \quad (26)$$

mit den Kontaktkräften aus (7). Eine statische Kondensation der Multiplikatoren ($\mathbf{P} = \mathbf{D}^{-1}\mathbf{M}$) ist auf diesem Branch bewusst nicht aktiviert.

12.2 Ein Newton pro Increment mit eingefrorener Geometrie

Material-, Geometrie- und Kontaktnichtlinearität werden in *einem* Newton-Verfahren pro Lastschritt gelöst. Wie in Abschnitt 10 beschrieben, werden Normalen und $\mathbf{D}$, $\mathbf{M}$ zu Increment-Beginn eingefroren; die assemblierte Tangente ist dann die exakte Jacobi-Matrix des Residuums, was die (nach Fixierung des Active Set) quadratische Konvergenz ermöglicht. Die konsistente Tangente ist per Finite-Differenzen zu $\sim 10^{-10}$ verifiziert (Abschnitt 13).

12.3 Active-Set-Iteration und Anti-Cycling

Das Active Set wird in *jeder* Newton-Iteration aus dem Indikator (24) neu bestimmt (`mortarcontact.py:797-839`). Pro Knoten gibt es zwei Zweige:

- **aktiv** ($s_{n,j} > 0$): Constraint-Residuum $\text{PExt}[\lambda] = \tilde{g}_j$; Slave-Kräfte $+\lambda_j \mathbf{D}[j, k] \mathbf{n}_j$, Master-Kräfte $-\lambda_j \mathbf{M}[j, l] \mathbf{n}_j$;
- **inaktiv** ($s_{n,j} \leq 0$): λ_j wird zu null gezwungen ($\text{PExt}[\lambda] = \lambda_j$, $K[\lambda, \lambda] = 1$).

Die äußere Iteration ist konvergiert, sobald sich das Set nicht mehr ändert (PDASS-Kriterium, Hübner & Wohlmuth, 2005). Gegen *Zyklen* (abwechselnde Active-Set-Zustände) sichert eine Anti-Cycling-Regel im Sinne von Bland: bei erneutem Besuch eines bereits gesehenen Zustands wird das Set eingefroren (`mortarcontact.py:841-867`); eine harte Obergrenze von 20 Iterationen dient als Sicherheitsnetz.

13 Verifikation

Die Implementierung ist durch eine Testsuite unter `testfiles/mortar_tests/` abgesichert; die folgenden Bausteine belegen die Korrektheit der obigen Herleitungen.

Eigenschaft	Prüfung
Polygon-Clipping	Schnitt-/Triangulierungsgeometrie gegen analytische Überlappungen (<code>02_polygon_clipping</code>).
Koppelmatrizen	D, M und Zeilensummen-Identität (10) numerisch (<code>03_coupling_matrices</code>).
Biorthogonalität	(11) numerisch verifiziert (<code>04_dual_biorthogonality</code>).
Quadratische Segmentierung	Sub-Cell-Zerlegung + Segmentquadratur für CONQUAD8/9, CONTRI6 (<code>05_quadratic_segmentation</code>).
Active-Set / PDASS	Aktivierung bei Durchdringung, Freeze/Anti-Cycling (<code>06_active_set_pdass</code>).
Patch-Test 2D	konstanter Druck exakt übertragen, CPE4/CPE8, konform + nichtkonform (<code>07_patch_test_2d</code>).
Patch-Test hex20	3D-Patch-Test mit CONQUAD8 exakt bestanden (<code>08_patch_test_hex20</code>).

Ergänzend gilt:

- **Kontakt-Patch-Test** (konstanter Druck exakt übertragen) in 2D (CPE4/CPE8) und 3D (hex8/hex20), konform *und* nichtkonform. Das bestätigt zugleich die Zeilensummen-Identität (10) und die korrekte Behandlung quadratischer Elemente (Basistransformation, Sub-Cells).
- **Konsistente Tangente** per Finite-Differenzen bestätigt ($\sim 10^{-10}$) – die Voraussetzung für die quadratische Newton-Konvergenz nach Fixierung des Active Set.
- **Zwei-Block-Referenztests** (`Control_Tests/README.md`): verschiebungs- und lastgesteuerte Blöcke, konform und nichtkonform, mit Abgleich von Verschiebungsfeld, Spannungen und Kontaktbedingungen.

Der reibungsfreie Mortar-Kontakt rechnet damit stabil – Increment für Increment bei vollem Lastschritt.

14 Forschung: Warum ist `hex20` im Ausziehversuch steifer als `hex8`?

Der Ausziehversuch eines Kopfbolzendübels aus einem Betonquader (`testfiles/mortar_tests/POT_Dejori`) wird mit demselben Modell einmal mit *linearen* Elementen (`hex8`: Beton `GC3D8`, Kontakt `CONQUAD4`) und einmal mit *quadratischen* Elementen (`hex20`: Beton `GC3D20R`, Kontakt `CONQUAD8`) gerechnet. Beide Netze sind bis auf die zusätzlichen Kantenmittelnknoten identisch (gleiche Elementzahl, dieselben sechs Kontaktflächen), das Material (gradientenerweiterte Schädigungsplastizität, `GCDP`) ist dasselbe, und beide Rechnungen sind verschiebungsgesteuert. Dennoch liefern sie deutlich verschiedene Kraft-Weg-Kurven. Da die Verifikation (Abschnitt 13) die Korrektheit der Implementierung für lineare *und* quadratische Elemente belegt, stellt sich die Frage: Wie kann dieser Unterschied zustande kommen?

14.1 Beobachtung

Der Unterschied ist in erster Linie eine *Steifigkeitsdifferenz*, und sie ist bereits im nahezu elastischen Bereich vollständig vorhanden – die Sekantensteifigkeit ist über die ersten 0,03 mm praktisch konstant, also *bevor* nennenswerte Schädigung eintritt und *bevor* das Active-Set aktiv wird.

	<code>hex8</code> (linear)	<code>hex20</code> (quadratisch)
Sekantensteifigkeit ($u \leq 0,03$ mm)	≈ 117 kN/mm	≈ 197 kN/mm
Peak-Last	22,6 kN	19,9 kN
Weg beim Peak	0,22 mm	0,12 mm

Im Vergleich mit den drei Laborversuchen trifft `hex20` sowohl die Anfangssteifigkeit als auch die Höhe des Peaks, während `hex8` zu weich ist und den Peak überschätzt. Es ist also nicht so, dass `hex20` zu steif wäre; vielmehr ist `hex8` zu weich, und genau das ist zu erklären. Weil die Differenz elastisch (vor Schädigung, vor Active-Set) auftritt, ist sie weder eine Frage des Schädigungsmodells noch der Kontakt-Aktivierung.

14.2 Der entscheidende Hinweis: die Steifigkeit bestimmt der Kontakt, nicht das Volumenelement

Bei einem *reinen* verschiebungsbasierten Finite-Elemente-Ansatz ist die diskrete Steifigkeit eine obere Schranke der wahren Steifigkeit, und ein reicherer Ansatzraum senkt sie: quadratische Elemente sind dort stets *weicher* und genauer als lineare. Wäre also allein der Betonkörper maßgeblich, müsste `hex20` weicher sein als `hex8` – niemals steifer. Auch die konkrete Integration der Volumenelemente zeigt in dieselbe Richtung: `GC3D8` ist voll integriert (und damit eher zu steif, versteifungsanfällig), `GC3D20R` ist reduziert integriert (und damit eher weicher). Beide Effekte auf der Volumenebene sagen $\text{hex8} \geq \text{hex20}$ voraus; beobachtet wird das Gegenteil.

Daraus folgt: Die zusätzliche Steifigkeit von `hex20` stammt aus der *Kontakt*-Lasteinleitung, nicht aus dem Volumenelement. Der Mortar-Kontakt ist eine *gemischte* (Lagrange-)Formulierung, in der die Monotonie “reicherer Raum = weicher” gerade nicht gilt; und das grobe lineare Netz ist an der Lasteinleitung erkennbar noch nicht konvergiert.

14.3 Der physikalische Mechanismus: Lasteinleitung am Bolzenkopf

Physikalisch ist der Ausziehversuch ein Lasteinleitungs- bzw. Stempelproblem: Der steife *Stahlkopf* presst sich gegen den Beton, und die Steifigkeit der Antwort hängt fast ausschließlich davon ab, wie gut diese *konzentrierte* Lasteinleitung am Kopf aufgelöst wird. Die lineare Diskretisierung versagt dabei an zwei Stellen gleichzeitig:

- **Randkonzentration des Kontaktdrucks.** Unter einem steifen Stempel steigt der Kontaktdruck zu den Rändern der Kontaktfläche hin stark an. Der lineare Mortar-Kontakt

(diagonale $\mathbf{D}$, Multiplikatoren nur an Eckknoten) kann diesen Randanstieg nicht darstellen und verschmiert den Druck; die Grenzfläche gibt dort zu leicht nach – die Antwort wird künstlich weich. Der quadratische Mortar-Kontakt (volle, nicht gelumpfte $\mathbf{D}$, zusätzliche Kantenmittelknoten, transformierte duale Basis nach Abschnitt 8; Popp et al., 2012) löst die Druckverteilung samt Randkonzentration deutlich besser auf.

- **Dehnungsgradient im Beton am Kopf.** Direkt am Kopf herrscht ein steiler Dehnungsgradient (hohe Pressung unter dem Kopf, rasch abklingend nach außen). Lineare Elemente verschmieren diesen Gradienten über das Element und mischen das hoch beanspruchte, steife Material am Kopf mit dem weicheren daneben zu einer zu nachgiebigen Mittelung; quadratische Elemente bilden den Gradienten ab.

Beides macht `hex8` zu weich und erklärt zugleich den überschätzten Peak: Weil die lineare Diskretisierung die lokale Spannungsspitze am Kopf unterschätzt, erreicht der Beton seine Festigkeit erst später; die Schädigung setzt verspätet ein, der Anker wird weiter gezogen, und die Kurve läuft auf einen höheren, späteren Peak. Die quadratische Diskretisierung erfasst die lokale Spitze korrekt, sodass die Schädigung bei der realen Last einsetzt und Peak-Höhe wie Peak-Lage zu den Versuchen passen.

14.4 Abgleich mit der Verifikation

Dies steht nicht im Widerspruch zu den bestandenen Patch- und Referenztests (Abschnitt 13). Diese prüfen gleichförmige Spannungs- bzw. Dehnungszustände an glatten Grenzflächen unter Verschiebungssteuerung – also gerade *ohne* Konzentration. In diesem Regime ist die Lösung elementordnungs*unabhängig* und der Mortar-Kontakt exakt, für lineare wie für quadratische Elemente. Der Ausziehversuch ist das gegenteilige Regime: Er ist konzentrationsdominiert, und genau dann entscheidet die Elementordnung. Beide Diskretisierungen sind korrekt; sie sind lediglich unterschiedlich gut aufgelöste Näherungen desselben, ordnungsempfindlichen Problems.

14.5 Schlussfolgerung

`hex20` ist die genauere, nahezu konvergierte Lösung und trifft die Laborversuche; `hex8` löst die konzentrierte Lasteinleitung am Bolzenkopf zu grob auf und ist deshalb künstlich weich, mit überschätztem Peak. Der große Unterschied zwischen den beiden Kurven ist damit ein Diskretisierungs- bzw. Elementordnungseffekt an der Lasteinleitung – kein Implementierungsfehler.

15 Zusammenfassung

Der auf Branch `0_mortarcontact` implementierte reibungsfreie Mortar-Kontakt überträgt Kräfte zwischen nichtkonformen Netzen über eine schwach (im integralen Mittel) durchgesetzte Nichtdurchdringungsbedingung. Die tragenden Bausteine sind:

- eine **segmentbasierte Integration** mit BVH-Nachbarsuche, Hilfsebenen-Projektion, Sutherland–Hodgman-Clipping und Grad-5-Quadratur (7-Punkt-Dreiecksregel nach [Dunavant 1985](#)), wobei der erhöhte Grad die nichtlineare Projektion auffängt ([Farah et al., 2015](#));
- **duale Lagrange-Multiplikatoren** ([Wohlmuth, 2000](#)), die $\mathbf{D}$ diagonalisieren und die Koppelmatrizen $\mathbf{D}, \mathbf{M}$ samt gewichtetem Spalt $\tilde{g}_j$ liefern, mit garantierter Zeilensummen-Identität (Impulserhaltung);
- eine **Basistransformation** für quadratische Elemente ([Popp et al., 2012](#); im Code der Farah-Wert $\alpha = \frac{1}{3}$) zur Sicherung positiver dualer Gewichte, plus lineare **Sub-Cells** fürs Clipping ([Puso et al., 2008](#));
- die **semiglatte NCP-Formulierung** der Signorini-Bedingung ([Gitterle et al., 2010](#), Gl. (55)) gelöst per **PDASS**/semismooth Newton ([Hüeber & Wohlmuth, 2005](#)).

Die Formulierung besteht die Kontakt-Patch-Tests in 2D und 3D (linear und quadratisch, konform und nichtkonform) und rechnet stabil bis in den entfestigenden Bereich des Betonmaterials. Reibung und statische Kondensation sind nicht Teil dieses Branches und wurden hier bewusst ausgeklammert.

16 Anwendungsanleitung: Kontaktspezifikation in der Input-Datei

16.1 Syntax-Struktur des Mortar-Constraint-Blocks

Der Kontakt wird im Modell-Header unter `*constraint` definiert. Auf Branch `0_mortarcontact` kennt der Constraint vier Argumente (`mortarcontact.py:89-108`): die beiden Pflicht-Oberflächen, das Feld und den NCP-Parameter `cn`:

```
1 *constraint, type=mortarcontact, name=mein_kontakt_name
2 nonMortarSurface=slave_surface_name
3 mortarSurface=master_surface_name
4 field=displacement
5 cn=30000.0
```

Listing 1: Mortar-Kontakt-Block (reibungsfrei) in der EdelweissFE Input-Datei

Der Wert von `cn` sollte in der Größenordnung des Elastizitätsmoduls des weicheren Kontaktpartners liegen (hier beispielhaft $\approx 3 \cdot 10^4$ für Beton in MPa); die genaue Bedeutung und Begründung stehen in Abschnitt 11.

16.2 Schlüsselwörter

Schlüsselwort	Beschreibung
<code>type=mortarcontact</code>	Pflichtangabe. Aktiviert den segmentbasierten Mortar-Kontakt-Constraint (2D und 3D).
<code>name=<string></code>	Eindeutiger Name des Kontaktpaars.
<code>nonMortarSurface=<string></code>	Pflicht. Name der Non-Mortar-Fläche (Slave) . Trägt die Kontaktknoten, Normalen $\mathbf{n}_I$ und dualen Formfunktionen Φ .
<code>mortarSurface=<string></code>	Pflicht. Name der Mortar-Fläche (Master) , auf die der Spalt gemessen wird.
<code>field=displacement</code>	Optional (Standard <code>displacement</code>). Verschiebungs-Vektorfeld, auf das der Kontakt wirkt.
<code>cn=<float></code>	Optional (Standard 10^6). Komplementaritätsparameter c_n der Normal-NCP (Abschnitt 11). <i>Sollte</i> pro Problem in der Größenordnung $\mathcal{O}(E)$ des weicheren Kontaktpartners gesetzt werden; der Default ist nur ein Rückfallwert.

Tabelle 1: Parameter des `mortarcontact`-Constraints auf Branch `0_mortarcontact`.

16.3 Vorbereitung der Oberflächen und Kontaktelemente

Der Mortar-Kontakt basiert auf expliziten Overlay-Kontaktelementen (`CONLINE2/3` in 2D bzw. `CONQUAD4/8/9`, `CONTRI3/6` in 3D). Beim Erzeugen aus Abaqus/Cubit-Netzen mit `prepare_mortar_mesh.py` gilt:

- **Normalenorientierung:** Elementnormalen müssen aus dem Kontinuumskörper nach außen weisen (`prepare_mortar_mesh.py` stellt dies automatisch sicher).
- **Slave/Master-Wahl:** Als Slave-Seite (`nonMortarSurface`) das feinere Netz bzw. das weichere Material wählen (Standardempfehlung der dualen Mortar-Methode).

Literatur

- T. Cichosz, M. Bischoff: *Consistent treatment of boundaries with mortar contact formulations using dual Lagrange multipliers*, Computer Methods in Applied Mechanics and Engineering 200 (2011), 1317–1332. (Lokale Kopie: meetings/literature/CichoszBischoff_2011_ConsistentTreatmentOfBoundariesWithMortarContactFormulationsUsingDualLagrangeMultipliers.pdf)
- D. A. Dunavant: *High degree efficient symmetrical Gaussian quadrature rules for the triangle*, International Journal for Numerical Methods in Engineering 21 (1985), 1129–1148. (Lokale Kopie: meetings/literature/Dunavant_1985_HighDegreeEfficientSymmetricalGaussianQuadratureRulesForTheTriangle.pdf)
- C. Ericson: *Real-Time Collision Detection*, Morgan Kaufmann, 2004. (Lokale Kopie: meetings/literature/Ericson_2004_RealTimeCollisionDetection.pdf)
- P. W. Farah: *Mortar Methods for Computational Contact Mechanics Including Wear and General Volume Coupled Problems*, Dissertation, TU München, 2018. (Lokale Kopie: meetings/literature/Farah_2018_MortarMethodsForComputationalContactMechanicsIncludingWearAndGeneralVolumeCoupledProblems.pdf)
- P. Farah, A. Popp, W. A. Wall: *Segment-based vs. element-based integration for mortar methods in computational contact mechanics*, Computational Mechanics 55 (2015), 209–228. (Lokale Kopie: meetings/literature/FarahPoppWall_2015_SegmentBasedVSElementBasedIntegrationForMortarMethodsInComputationalContactMechanics.pdf)
- M. Gitterle, A. Popp, M. W. Gee, W. A. Wall: *Finite deformation frictional mortar contact using a semi-smooth Newton method with consistent linearization*, International Journal for Numerical Methods in Engineering 84 (2010), 543–571. (Lokale Kopie: meetings/literature/GitterlePoppGeeWall_2010_FiniteDeformationFrictionalMortarContactUsingASemiSmoothNewtonMethodWithConsistentLinearization.pdf)
- M. Hintermüller, K. Ito, K. Kunisch: *The primal-dual active set strategy as a semismooth Newton method*, SIAM Journal on Optimization 13 (2002), 865–888. (Lokale Kopie: meetings/literature/HintermuellerItoKunisch_2002_ThePrimalDualActiveSetStrategyAsASemismoothNewtonMethod.pdf)
- S. Hüeber, B. I. Wohlmuth: *A primal-dual active set strategy for non-linear and contact problems*, Computer Methods in Applied Mechanics and Engineering 194 (2005), 3147–3166. (Lokale Kopie: meetings/literature/HueberWohlmuth_2005_APrimalDualActiveSetStrategyForNonLinearContactProblems.pdf)
- B. P. Lamichhane, R. P. Stevenson, B. I. Wohlmuth: *Higher order mortar finite element methods in 3D with dual Lagrange multiplier bases*, Numerische Mathematik 102 (2005), 93–121. (Lokale Kopie: meetings/literature/LamichhaneStevensonWohlmuth_2005_HigherOrderMortarFiniteElementMethodsIn3DWithDualLagrangeMultiplierBases.pdf)
- A. Popp, M. W. Gee, W. A. Wall: *A finite deformation mortar contact formulation using a primal-dual active set strategy*, International Journal for Numerical Methods in Engineering 79 (2009), 1354–1391. (Lokale Kopie: meetings/literature/PoppGeeWall_2009_AFiniteDeformationMortarContactFormulationUsingAPrimalDualActiveSetStrategy.pdf)
- A. Popp, M. W. Gee, W. A. Wall: *A dual mortar approach for 3D finite deformation contact with consistent linearization*, International Journal for Numerical Methods in Engineering 83 (2010), 1428–1465.

- A. Popp, B. I. Wohlmuth, M. W. Gee, W. A. Wall: *Dual quadratic mortar finite element methods for 3D finite deformation contact*, SIAM Journal on Scientific Computing 34 (2012), B421–B446. (Lokale Kopie: `meetings/literature/PoppWohlmuthGeeWall_2012_DualQuadraticMortarFiniteElementMethodsFor3DFiniteDeformationContact.pdf`)
- M. A. Puso, T. A. Laursen: *A mortar segment-to-segment contact method for large deformation solid mechanics*, Computer Methods in Applied Mechanics and Engineering 193 (2004), 601–629. (Lokale Kopie: `meetings/literature/PusoLaursen_2004_AMortarSegmentToSegmentContactMethodForLargeDeformationSolidMechanics.pdf`)
- M. A. Puso, T. A. Laursen, J. Solberg: *A segment-to-segment mortar contact method for quadratic elements and large deformations*, Computer Methods in Applied Mechanics and Engineering 197 (2008), 555–566. (Lokale Kopie: `meetings/literature/PusoLaursenSolberg_2008_ASegmentToSegmentMortarContactMethodForQuadraticElementsAndLargeDeformations.pdf`)
- I. E. Sutherland, G. W. Hodgman: *Reentrant polygon clipping*, Communications of the ACM 17 (1974), 32–42. (Lokale Kopie: `meetings/literature/SutherlandHodgman_1974_ReentrantPolygonClipping.pdf`)
- B. I. Wohlmuth: *A mortar finite element method using dual spaces for the Lagrange multiplier*, SIAM Journal on Numerical Analysis 38 (2000), 989–1012. (Lokale Kopie: `meetings/literature/Wohlmuth_2000_AMortarFiniteElementMethodUsingDualSpacesForTheLagrangeMultiplier.pdf`)
- P. Wriggers: *Computational Contact Mechanics*, 2. Aufl., Springer, 2006. (Lokale Kopie: `meetings/literature/Wriggers_2006_ComputationalContactMechanics.pdf`)